

PRACTICE EXAM 4 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 4

The last twenty. Eighty questions in total, and the trend across all four is what matters.

QUESTIONS 20 RUNNING TOTAL 80 PAUSE 9 seconds

Last paper. What it is for.

- ◆ **This is a check, not a diagnosis.** The diagnosis came from papers one to three.
- ◆ **Sit it under proper conditions if you can.** Timed, no notes, no pausing.
- ◆ **Have all four tallies to hand at the end.** The closing slides read across them.
- ◆ **If a previously weak domain holds up here, it is fixed.** That is what you are testing.

ON THE EXAM

Eighty questions across four sittings tells you something a single mock exam cannot.

An agent processes user-submitted content. Occasionally a response comes back with `stop_reason` refusal and no tool calls.

The orchestration layer treats any run without a completed task as a transient failure and retries the identical request three times, then logs an error.

Costs on those items are four times normal and the outcome never changes.

What should the wrapper do instead?

- ☐ A Retry with exponential backoff so the model has time to recover
- ☐ B Treat refusal as a terminal outcome: do not auto-retry, log it, and surface an appropriate message
- ☐ C Rephrase the request automatically and resubmit
- ☐ D Increase `max_tokens`, since refusals often follow truncation

◆ PAUSE AND DECIDE

What should the wrapper do instead?

- A

 Backoff
Backoff is for transient failures. A refusal is a decision, not a blip.
- C

 Auto-rephrase
Automatically reworking a request until it is not refused is the wrong instinct to build in.
- B

 Terminal outcome
refusal arrives as a normal successful response. Retrying identically changes nothing.
- D

 More output tokens
Unrelated. Refusal is not truncation.

THE SIGNAL

refusal is one of the six ways a run ends, and it is not an error. Classify it, do not retry it.
Module 4.

A platform team maintains a release workflow used identically across fourteen service repositories. It consists of four skills, a PreToolUse hook, and an MCP server configuration.

Today it is copy-pasted into each repository. Updates require fourteen pull requests and five repositories are already out of date.

What should they do?

- ☐ A Move it to user-level configuration so each engineer has it everywhere
- ☐ B Package it as a plugin and install it in each repository
- ☐ C Put it in a shared repository and import it with @path from each CLAUDE.md
- ☐ D Move it into the enterprise managed settings file

◆ PAUSE AND DECIDE

What should they do?

- A

 User-level config
Per engineer, not per repository. Every joiner sets it up by hand.
- B

 Package as a plugin
A plugin is exactly a bundle of skills, hooks and MCP config, with updates handled.
- C

 Shared repo plus imports
Imports pull in memory file content. They do not distribute hooks or MCP configuration.
- D

 Enterprise managed settings
That layer is for policy that must not be overridden. This is a workflow.

THE SIGNAL

The contents list is the answer: skills plus a hook plus MCP config is the definition of a plugin. Module 9.

A defect triage system classifies incoming reports using an enum of eight defect types, defined as a closed set.

An audit finds that novel defect types are consistently assigned to whichever of the eight is nearest. A new failure mode in a recently shipped component ran for five weeks classified as "configuration" before anyone noticed.

The team wants to catch new types earlier.

What is the most effective change?

- ☐ A Add enum members for the defect types they know about now
- ☐ B Add an 'other' member with a required free-text description, and review those weekly
- ☐ C Add a confidence score and review low-confidence classifications
- ☐ D Remove the enum and allow free-text defect types

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Add known members
Fixes the types that already surfaced. The next novel one has the same blind spot.
- B

 'other' plus description
Novel defects become visible as a category rather than hiding inside an existing one.
- C

 Confidence scores
A strong answer elsewhere, and it depends on the model being unsure. It confidently picked configuration.
- D

 Free text
Removes the enum's guarantee. Miscategorisation traded for unbounded drift.

THE SIGNAL

"Consistently assigned to whichever is nearest" means it was not uncertain. A closed enum forces a wrong choice. Module 13.

A long-running research agent approaches its context limit. Almost all of the context is raw tool results from document fetches made much earlier, whose conclusions have already been recorded in the agent's notes.

The team wants to free space with the least risk to the reasoning so far, and without paying for an extra model call.

What is the most appropriate mechanism?

- ☐ A Compaction: summarise the whole transcript and reinitiate with the summary
- ☐ B Tool result clearing: drop the stale tool calls and their results
- ☐ C Start a fresh session with an injected summary
- ☐ D Move to a model with a larger context window

◆ PAUSE AND DECIDE

What is the most appropriate mechanism?

- A

Compaction
Costs an inference call and rewrites the whole transcript, losing verbatim detail everywhere.
- B

Tool result clearing
Targets exactly the stale material, costs no inference, and leaves the reasoning untouched.
- C

Fresh session
Heavier than needed. The reasoning is fine; only the tool results are dead weight.
- D

Bigger window
Buys headroom at the same fill rate.

THE SIGNAL

Clearing is the lightest touch: no inference cost, and it drops only what you name.
Module 15.

A helpdesk receives four clearly distinct request types: password resets, software installation, hardware faults and access requests. Each has its own well-tested handling prompt, and the four differ substantially.

Currently one large prompt covers all four. Quality on password resets has dropped since the access-request instructions were expanded.

The types are easy to tell apart from the message.

What is the most appropriate structure?

- ☐ A One prompt with all four sets of instructions and stronger separation between them
- ☐ B A classification step that routes to one of four specialised prompts
- ☐ C An agent with four tools, one per request type
- ☐ D Four parallel calls, one per specialised prompt, taking the best answer

◆ PAUSE AND DECIDE

What is the most appropriate structure?

- A

 One bigger prompt
Four job descriptions competing. Expanding one already degraded another.
- B

 Route to specialised prompts
Distinct, separable categories where tuning one path must not affect the others.
- C

 Four tools
Keeps everything in one context, so the instruction sets still compete.
- D

 Four parallel calls
Four times the cost, and it needs a judge to pick a winner.

THE SIGNAL

Distinct categories, and tuning one path must not affect the others. That is routing.
Module 2.

A team wants to integrate with a widely used CRM. The vendor publishes a maintained MCP server handling authentication and pagination, exposing thirty-four tools.

An engineer proposes building an in-house server, arguing they need only five operations and thirty-four tools will hurt selection accuracy and cost.

The team has no capacity for ongoing maintenance.

What should you advise?

- ☐ A Build in-house; five focused tools will outperform thirty-four overlapping ones
- ☐ B Adopt the vendor's server and restrict which of its tools the agent may use
- ☐ C Adopt the vendor's server as-is; more tools give the agent more options
- ☐ D Build in-house and contribute it back to the vendor

◆ PAUSE AND DECIDE

What should you advise?

- A

 Build in-house
The tool-count argument is sound, and the stem says there is no maintenance capacity.
- B

 Adopt and restrict
Keeps the vendor's auth and fixes, and gets a five-tool surface by allowing only what is needed.
- C

 Adopt as-is
Ignores a real problem. Thirty-four definitions tax every request and worsen selection.
- D

 Build and contribute
All the maintenance burden of A, plus an open-source commitment.

THE SIGNAL

The engineer's diagnosis is right and the prescription is wrong. Permissions give a narrow toolset without owning the server. Module 7.

An engineer asks Claude Code to add audit logging across a service they have never worked in. The change will touch the middleware, the configuration schema, five handlers and the test suite.

Claude begins editing immediately. Half an hour later the engineer finds the logging was added in the wrong layer and the work must be unpicked.

This pairs with exam 1 question 14, where skipping the plan was correct.

What should they change?

- ☐ A Add a line to CLAUDE.md telling Claude to propose an approach before making changes
- ☐ B Use plan mode for changes of this shape and approve the plan before implementation
- ☐ C Ask Claude to explain its approach in the prompt before it starts
- ☐ D Add a PreToolUse hook blocking Edit until a plan file exists

◆ PAUSE AND DECIDE

What should they change?

- A

 A line in CLAUDE.md
Context. It works most of the time, which is what they already have.
- B

 Plan mode
Unfamiliar code, several files, an arguable approach. Edits are blocked until approval.
- C

 Ask in the prompt
A request with no gate. It explains, then implements anyway.
- D

 Hook blocking Edit
Enforcement aimed badly. It blocks writes mid-task and breaks multi-step reasoning.

THE SIGNAL

The pair: exam 1 was a one-line change, so skip the plan. This is unfamiliar code across five files, so plan mode. Module 10.

An agent's loop appends tool results to the conversation. To help the model, the implementation also adds a short note alongside each tool result, such as "Here are the results you asked for."

Over long conversations the agent increasingly returns empty responses with `stop_reason` `end_turn`, having done nothing.

The tool results themselves are correct and well formed.

What is the most likely cause?

- ☐ A The context window is filling and the model has nothing left to work with
- ☐ B The extra text teaches the model to expect user input after every tool call, so it waits instead of continuing
- ☐ C The tool results are being appended without the preceding assistant turn
- ☐ D `max_tokens` is too low for the responses

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Context filling
Would produce vaguer answers, not empty ones with a clean end_turn.
- B

 Trailing text after tool results
The user turn following tool_use should contain tool_result blocks only.
- C

 Missing assistant turn
That produces an API error immediately, not empty responses.
- D

 max_tokens too low
That gives truncated output with stop_reason max_tokens, not empty output with end_turn.

THE SIGNAL

The user turn after tool_use carries tool_result blocks and nothing else. Helpful extra text is the bug. Module 1.

A new extraction service asks the model to "return the result as JSON matching this structure" and pastes an example object into the prompt.

At low volume it works. In a load test of 40,000 documents, around 700 responses could not be parsed: markdown fences, a preamble sentence, and a handful of renamed fields.

The team is deciding what to do before launch.

What is the most appropriate change?

- ☐ A Add a regex cleanup step stripping fences and preamble before parsing
- ☐ B Define the target shape as a tool input_schema and force it with tool_choice
- ☐ C Add "return only JSON, no other text" to the prompt in capitals
- ☐ D Retry any unparseable response up to three times

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

Regex cleanup
Handles fences and preamble. It cannot rescue a renamed field, and it is a parser you maintain.
- B

Tool schema, forced
Moves the shape out of prose into a structure the model is trained to respect.
- C

Capitals
Emphasis is not a technique.
- D

Retry unparseable
Treats a design problem as transient. The rate stays about the same.

THE SIGNAL

Works at low volume, falls apart under load, is how prompt-only JSON always presents. Module 13.

A scheduling tool rejects requests that fall outside business hours. On rejection it returns `isError` with the message "Error: invalid request."

Production traces show the agent abandoning the booking task entirely after one rejection, telling the user scheduling is unavailable.

The tool works correctly for in-hours requests.

What is the most effective change?

- ☐ A Return a message naming the constraint, the business hours, and the nearest valid slot
- ☐ B Retry the same request up to three times before giving up
- ☐ C Report the rejection as a protocol error so the agent treats it as fatal
- ☐ D Instruct the agent in the system prompt not to give up after a tool error

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Actionable message
Names the field, the rule and the recovery path, so the agent can retry with a valid time.
- B

 Retry the same request
A deterministic rejection. The same time is outside hours on every attempt.
- C

 Protocol error
Makes it worse. The agent would see even less.
- D

 Instruct it not to give up
Abandoning is the rational response to an error with no detail in it.

THE SIGNAL

Write the error as an instruction. Accuracy is not the standard; actionability is. Module 6.

A coordinator spawns four subagents to investigate a production incident, aggregates their findings, and produces a report.

Reviewers note that reports frequently identify a promising lead — an unusual error pattern in one service — and then stop, because no subagent was tasked with following it.

The four subagents are chosen at the start and never change.

What is missing from the design?

- ☐ A More subagents at the start, to cover more ground
- ☐ B A coordinator step that reads the results, identifies gaps, and spawns further subagents
- ☐ C Direct communication between subagents so leads can be handed over
- ☐ D A larger context window for the coordinator

◆ PAUSE AND DECIDE

What is missing from the design?

- A

 More subagents upfront
You still cannot name the lead before the first round finds it.
- B

 Gap-driven re-delegation
Round two depends on round one. That is what makes it orchestration rather than parallelisation.
- C

 Subagent messaging
Invented. Subagents have no channel to each other.
- D

 Bigger coordinator window
Nothing here is about capacity.

THE SIGNAL

Fire once and aggregate is parallelisation. Reading results and re-delegating is orchestration. Module 3.

A long-running analysis agent uses compaction when it nears the context limit. It works, and the overall conclusions survive intact.

Analysts report that specific figures cited earlier — a particular confidence interval, a footnoted exclusion — go missing after a compaction.

Those figures are needed at the end of the analysis.

What is the most appropriate response?

- ☐ A Raise the compaction threshold so it fires less often
- ☐ B Write key figures and caveats to a file as they are found, so they survive outside the transcript
- ☐ C Disable compaction and accept hitting the context limit
- ☐ D Instruct the compaction prompt to preserve everything

◆ PAUSE AND DECIDE

What is the most appropriate response?

- A

 Higher threshold
Fewer compactions, same loss each time. It delays the problem.
- B

 Write them outside the transcript
What is not in the transcript cannot be compacted away.
- C

 Disable compaction
Trades a lossy session for one that stops entirely.
- D

 Preserve everything
A summary that preserves everything is the transcript. Compaction that does not compress is not compaction.

THE SIGNAL

Conclusions survive, specifics are lost, is compaction working correctly. You need a different mechanism. Module 15.

A skill runs a database migration against the staging environment. It has side effects and takes several minutes.

Engineers report that Claude sometimes invokes it unprompted when someone asks a question like "would this migration work on staging?"

The team still wants to run it themselves with a slash command.

Which frontmatter field?

- | | |
|--|---|
| ☐ A user-invocable: false | ☐ B disable-model-invocation: true |
| ☐ C allowed-tools, restricted to read tools | ☐ D context: fork |

◆ PAUSE AND DECIDE

Which frontmatter field?

- A

`user-invocable: false`
The reversal. Removes the slash command and leaves Claude as the only invoker.
- B

`disable-model-invocation`
Claude can no longer trigger it; the slash command stays. Both requirements, one field.
- C

`allowed-tools`
Restricts what the skill may do, not who may start it.
- D

`context: fork`
Isolates context. Nothing to do with invocation.

THE SIGNAL

Two similarly named fields, opposite effects. Read which invoker the stem wants removed. Module 9.

An agent extracts terms from lease documents through a single tool with a large structured input. On long leases the run fails.

The response returns `stop_reason max_tokens` and the last content block is a partial `tool_use` whose input is cut off mid-object. The retry logic resends the identical request three times, failing the same way every time.

Some leases are far longer than any reasonable output budget.

What is the most effective fix?

- ☐ A Add exponential backoff between retries
- ☐ B Catch the parse error and have the agent summarise the lease in prose instead
- ☐ C Raise `max_tokens`, and split very long leases across several tool calls
- ☐ D Reduce the number of tools so more budget is left for output

◆ PAUSE AND DECIDE

What is the most effective fix?

- A

 Backoff
Deterministic failure. The same input overflows the same budget every time.
- B

 Fall back to prose
Abandons the structured output the pipeline needs. A hard failure becomes a silent one.
- C

 Raise the budget and split
Raising helps the near misses; splitting handles leases no budget would fit.
- D

 Fewer tools
Tool definitions are input. max_tokens caps output. Right instinct, wrong limit.

THE SIGNAL

"Far longer than any reasonable budget" is why raising alone is not enough. Both halves of C matter. Module 14.

An incident investigation agent spawns three subagents to read logs, query metrics and search runbooks. It works well.

The team wants to reduce cost. Their proposal is to run the three subagents concurrently rather than sequentially, expecting to cut the bill by about two thirds.

Latency is not a problem; investigations run overnight.

What should you tell them?

- ☐ A Concurrency will not reduce cost. Consider a cheaper model for the subagents, or narrower briefs that read less
- ☐ B The estimate is roughly right, since concurrent execution shares context loading
- ☐ C Concurrency reduces cost only if the subagents share tools
- ☐ D Merge the three subagents into one to eliminate two context loads

◆ PAUSE AND DECIDE

What should you tell them?

- A

 No cost benefit; here are two levers
Correct on both halves. Tokens are unchanged by scheduling.
- B

 Shared context loading
Invented. Each subagent loads its own context whether it runs now or later.
- C

 Only with shared tools
Tool overlap has no bearing on billing. Plausible-sounding and made up.
- D

 Merge into one
Saves two context loads and gives up the isolation. The stem never said the design was wrong.

THE SIGNAL

"Latency is not a problem" tells you the one thing concurrency buys is worthless here. Module 3.

A site-reliability team uses Claude Code with an MCP server for their alerting platform. During every incident an engineer types the same long request: pull the last two hours of alerts for a named service, group by severity, correlate with recent config changes, and format as an incident note.

The wording varies between engineers and the output format is inconsistent.

What is the most appropriate solution?

- ☐ A Add a tool to the server performing the whole sequence
- ☐ B Publish an MCP prompt taking the service name, so it appears as a slash command
- ☐ C Add the standard wording to CLAUDE.md so everyone phrases it the same way
- ☐ D Expose the incident note format as an MCP resource

◆ PAUSE AND DECIDE

What is the most appropriate solution?

- A

 One big tool
Plausible, and it hard-codes an investigation engineers vary deliberately.
- B

 An MCP prompt
A user-invoked template with a parameter is exactly this shape. One slash command, same structure.
- C

 Wording in CLAUDE.md
Context, not a command. Engineers still have to remember the phrasing.
- D

 Format as a resource
Fixes format consistency and none of the typing.

THE SIGNAL

A human repeating something wants a prompt. A model repeating something wants a tool. Module 7.

A team is launching a document classification service. They plan to route anything below 0.8 confidence to human review, having chosen 0.8 because it sounded reasonable.

They have 500 labelled documents from a pilot with known correct classifications, which they have not used.

Reviewer capacity is limited and they do not know whether 0.8 will overwhelm it.

What should they do first?

- ☐ A Launch with 0.8 and adjust based on reviewer feedback
- ☐ B Run the pilot set through the system and plot errors against confidence to choose the threshold
- ☐ C Set the threshold to whatever produces a review volume matching capacity
- ☐ D Route everything to review for the first month

◆ PAUSE AND DECIDE

What should they do first?

- A

 Launch and adjust
Reviewer feedback tells you about volume, not about what is slipping past.
- B

 Use the labelled set
They already have the data that turns a guess into a decision.
- C

 Fit to capacity
Sets a quality control by staffing level. It may sit far from where the errors are.
- D

 Review everything
Safe, unsustainable, and it never produces the threshold.

THE SIGNAL

"500 labelled documents, which they have not used" is the planted detail. The answer is on their disk. Module 17.

A nightly job runs `claude -p` in a CI container to triage failing tests, invoked with `--allowedTools Read,Grep,Glob,Bash`.

A security review raises two concerns. The runner's environment contains deployment credentials, and the job has previously run a Bash command that read a `.env` file into context.

The job genuinely needs to run the test suite.

What should they do?

- ☐ A Remove Bash from `--allowedTools`
- ☐ B Deny reading `.env` files, scrub sensitive environment variables from the subprocess, and keep Bash scoped to the test command
- ☐ C Add a `CLAUDE.md` rule instructing Claude never to read credential files
- ☐ D Use `--dangerously-skip-permissions` since the container is ephemeral

◆ PAUSE AND DECIDE

What should they do?

- A

 Remove Bash
Removes the capability the job exists to use. The stem says it needs to run tests.
- C

 A CLAUDE.md rule
Context. It cannot stop a Bash command reading a file.
- B

 Deny, scrub, and scope
Three enforced layers, and the job keeps the capability it needs.
- D

 Skip permissions
Backwards. Ephemeral limits blast radius; it does nothing about credentials in the environment.

THE SIGNAL

Ephemeral is not the same as safe. The credentials are present while the job runs, which is when the leak happens. Module 11.

An engineer asks an agent to "tidy up the reporting queries". It spends twenty minutes reformatting SQL, renaming aliases and reordering clauses.

None of it was wanted. They had meant "remove the four queries nobody runs any more".

This pattern recurs across many different open-ended requests from many different users.

What is the best general fix?

- ☐ A Add a CLAUDE.md section defining what "tidy up" means for this team
- ☐ B Have the agent state its assumptions about scope and format before producing long output
- ☐ C Have the agent ask three clarifying questions before every request
- ☐ D Add few-shot examples of good tidy-up work

◆ PAUSE AND DECIDE

What is the best general fix?

- A

 Define "tidy up"
Fixes this phrase. The next vague request is a different phrase with the same failure.
- B

 State assumptions first
Generalises across request types, costs a line, and lets the user correct before the work is done.
- C

 Three questions every time
Interrogation. Most requests are not ambiguous, and the friction lands on all of them.
- D

 Examples of tidy-up
Same limitation as A. It teaches one output shape for one request type.

THE SIGNAL

"Many different open-ended requests from many different users" rules out every fix specific to one phrase. Module 12.

An integration exposes thirty-eight tools, generated automatically one per REST endpoint. Many overlap: `get_account`, `get_account_summary` and `fetch_account_details` all return overlapping data.

The agent frequently selects a tool returning almost the right information, then calls another for the missing field. Token costs are high even on simple requests.

One engineer proposes writing much more detailed descriptions for all thirty-eight.

What is the most effective approach?

- ☐ A Write the detailed descriptions, since the description is the interface
- ☐ B Identify the workflows the agent actually performs and build a small set of tools for them
- ☐ C Keep all thirty-eight but instruct the agent to prefer `get_account`
- ☐ D Split the thirty-eight tools across several MCP servers to reduce per-server load

◆ PAUSE AND DECIDE

What is the most effective approach?

- A

 Better descriptions for all 38
Descriptions cannot separate tools that genuinely overlap, and 38 definitions still tax every request.
- C

 Prefer `get_account`
One hard-coded preference among thirty-eight overlapping tools.
- B

 Build for the workflows
Fixes overlap, chaining and token cost together, because all three come from porting the API.
- D

 Split across servers
All definitions still load into the same context. Reorganising the shelf does not shrink the library.

THE SIGNAL

One tool per REST endpoint is the ported-API anti-pattern. Start from the workflows. Module 5.

Reading four papers, not one score

- ◆ **Lay the four domain tallies side by side.** Eighty questions, five rows, four columns.
- ◆ **A row weak in all four is the one thing left to do.** Work its scenarios, not its slides.
- ◆ **A row that improved and stayed improved is finished.** Do not revisit it.
- ◆ **Rows that move around are noise.** Twenty questions per paper cannot resolve a small difference.
- ◆ **Do not average the four into a percentage.** The real exam is scaled, and your papers are not it.

ON THE EXAM

Four sittings of twenty is a far better instrument than one sitting of eighty would have been.

Which domain did you miss?

QUESTION NUMBERS	DOMAIN	MODULES TO REWATCH
1, 5, 8, 11, 15	D1 Agentic architecture	1 to 4
6, 10, 16, 20	D2 Tool design and MCP	5 to 7
2, 7, 13, 18	D3 Claude Code	8 to 11
3, 9, 14, 19	D4 Prompt and structured output	12 to 14
4, 12, 17	D5 Context and reliability	15 to 17

ON THE EXAM

Across all four papers: 22 questions in D1, 14 in D2, 16 in D3, 16 in D4, 12 in D5. Within a point of the real weights.

That is the course

- ◆ **Twenty five videos, five domains, 186 original questions.** None of them from a leaked bank.
- ◆ **Before you book:** check the current fee, the access rules, and whether a newer exam guide exists.
- ◆ **On the day:** read the question line first, commit before reading the options, answer everything.
- ◆ **Prompts persuade. Code enforces.** If you remember one line from the whole course, make it that one.
- ◆ **Good luck.**

ON THE EXAM

Module 20 lists the five places where the exam's answer and current documentation diverge. Worth one last pass.